LLMOPS / WEEK 06

# 지도 미세조정과 메모리 효율

SFT · LoRA · QLoRA

6주차 학습 단원 · 7주차 통합 강의 복습

차성재 · 무신사 Core AI PM × AI 대학원 겸임교수

# 6–7주차 자료의 연결

## 10월 9일 휴일에는 수업·녹화·신규 과제 없음

6주차 기술 내용은 10월 16일 통합 강의에서 진행

통합 녹화와 단계별 실습을 함께 확인

기획서 마감: 10.22(목) 23:59 한국시간 · 10.23은 제출물 평가

# 학습 내용의 세 축

| | | |
|---|---|---|
| 01 | SFT | 어떤 입력과 기대 답변으로 학습하는가 |
| 02 | LoRA | 어떤 파라미터를 갱신하는가 |
| 03 | QLoRA | 고정 모델을 어떤 정밀도로 저장하는가 |

지도 미세조정이라는 학습 목적에 LoRA 또는 QLoRA를 결합

# 100분 복습 진행 예시

| 구간 | 주요 내용 | 권장 시간 |
| --- | --- | --- |
| **배경** | 문제와 기술 선택 | 10분 |
| **SFT** | 데이터·손실·마스킹 | 25분 |
| **LoRA** | 행렬·설정·비용 | 25분 |
| **QLoRA** | 정밀도·메모리 | 20분 |
| **실습 정리** | 실행·비교·평가 계획 | 20분 |

자가 복습용 시간 배분 · 코드 예시는 해당 개념을 읽으며 실행

# 모델 적응의 배경

학습한 언어 패턴과 업무 규칙의 차이

먼저 실패 유형을 확인한 뒤 학습 범위 결정

# 사전학습과 지도 미세조정

## 사전학습 모델

대규모 텍스트의 통계적 패턴 학습

업무별 출력 규칙은 추가 검증

## 지도 미세조정 모델

입력과 기대 답변 쌍으로 추가 학습

반복 형식·행동의 적응에 활용

# 학습과 사용 단계의 차이

01

## 학습 입력

기대 답변 포함

02

## 손실 계산

예측과 정답 비교

03

## 가중치 갱신

학습 대상만 변경

04

## 추론

입력에서 답변 생성

일반적인 추론 요청은 모델 가중치를 갱신하지 않음

# 프롬프트·검색·미세조정의 비교

| 방법 | 바꾸는 대상 | 먼저 확인할 조건 |
| --- | --- | --- |
| **프롬프트** | 요청의 지시·예시 | 같은 입력의 오류 |
| **검색 증강 생성** | 요청에 제공할 문서 | 최신성·접근 권한 |
| **지도 미세조정** | 모델의 응답 패턴 | 허가된 예시·평가셋 |

검색 증강 생성: Retrieval-Augmented Generation, RAG · 함께 적용 가능

# 사례 A · 바뀐 정책의 오답

## 정책은 바뀌었지만 모델은 이전 조건으로 답변

먼저 문서 버전·검색 결과·적용일 점검

정책 원문을 학습에만 넣으면 갱신·삭제가 어려움

합성 사례 · 최신 지식 제공과 반복 행동 학습의 구분

# 사례 B · 반복되는 형식 오류

## 현재 대안

프롬프트에 JSON 예시 제공

필수 키 검사와 재시도 적용

## 미세조정 검토

동일한 실패가 반복되는지 기록

정상·예외·보류 답변을 함께 준비

JSON: JavaScript Object Notation · 데이터 교환 형식

# 미세조정 검토 순서

01

## 실패 수집

입력·원문 보존

02

## 기준선

프롬프트 대안

03

## 자료 준비

허가·분할

04

## 비교 평가

품질·운영 비용

변경할 대상과 성공 조건을 먼저 정한 뒤 실행

# 실험 가설의 기록

| 항목 | 수업용 제안 예시 |
| --- | --- |
| **고칠 문제** | 필수 JSON 키가 반복 누락 |
| **변경 범위** | 동일 Base에 LoRA 적용 |
| **비교 조건** | 동일 입력·문서·생성 설정 |
| **성공·중단** | 형식 개선과 근거 오류를 함께 검토 |

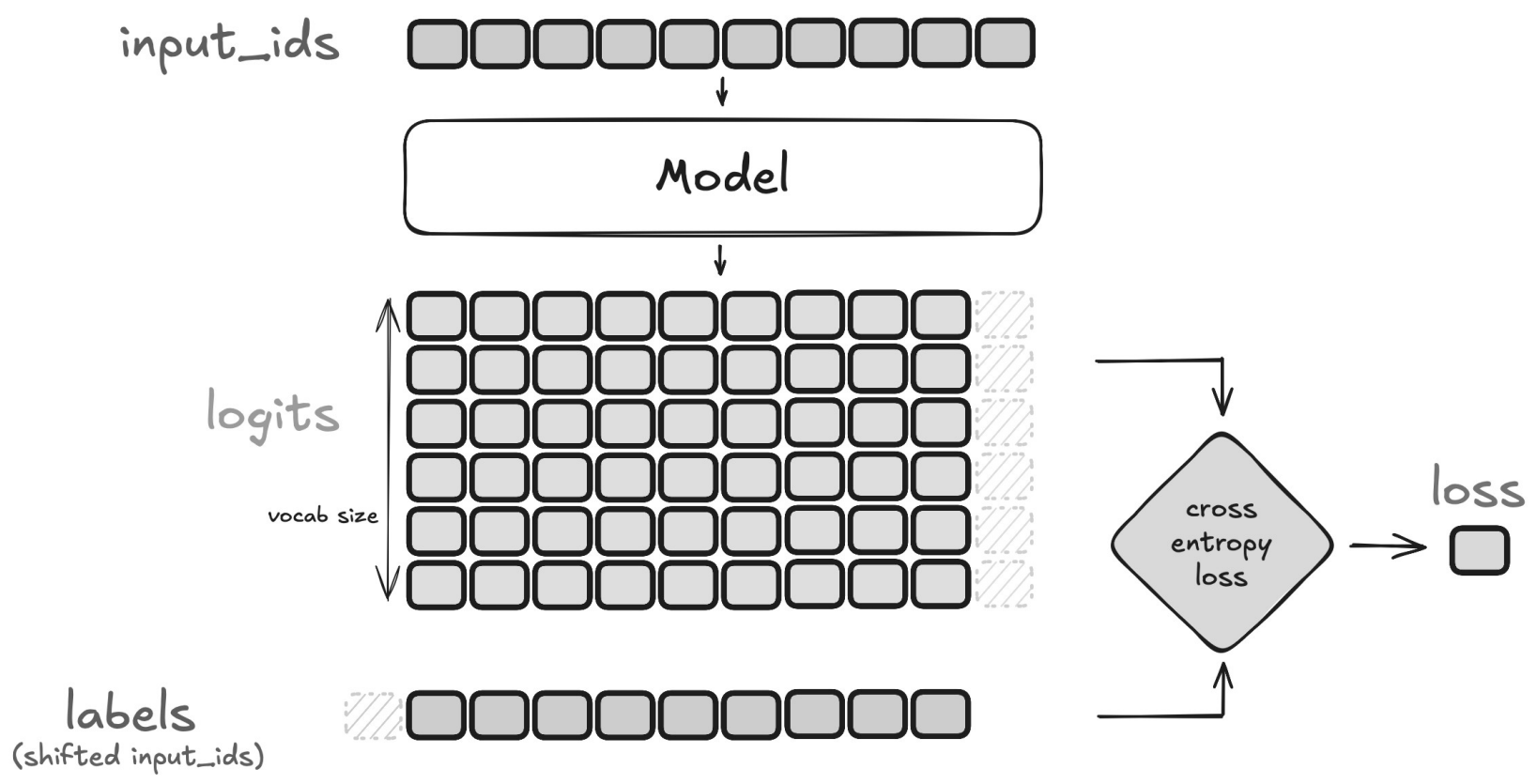
목표와 가설 예시 · 이미 달성한 결과를 뜻하지 않음

# 지도 미세조정

Supervised Fine-Tuning · SFT

업무 입력과 기대 답변으로 다음 토큰 예측 학습

# SFT의 토큰 예측과 손실




## 입력

input_ids: 토큰 ID의 나열

## 예측

logits: 다음 토큰의 점수

## 비교

labels와 교차 엔트로피 손실

Hugging Face TRL 공식 도식 · 원본 이미지의 흑백 표시

# 학습 데이터 한 건

```
instruction: 문의 상태 정리
input: 합성 사례 DEMO-01
response:
  status: review
  source: DEMO-01
```

## 입력과 기대 출력의 경계

합성 사례만 사용

형식과 내용의 정확성 별도 검토

실제 고객·정책 정보가 아닌 교육용 예시

# 대화 형식과 템플릿

01

## 역할

user·assistant

02

## 직렬화

모델별 구분자

03

## 토큰화

문자열을 ID로

04

## 학습 경계

답변·종료 토큰

Chat template: 대화를 해당 모델의 입력 문자열로 바꾸는 규칙

# 토큰과 문자 길이

## 문자 길이

공백·한글·기호의 문자 수

len(text)로 확인

## 토큰 길이

모델 토크나이저의 분할 결과

같은 문장도 모델에 따라 달라짐

수업용 가상 ID를 실제 tokenizer 결과로 해석하지 않음

# 다음 토큰의 정답 정렬

| 예측 위치 | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 입력 토큰 | 10 | 11 | 12 | 20 | 21 |
| 다음 정답 | 11 | 12 | 20 | 21 | 22 |
| 답변 학습 | 제외 | 제외 | 사용 | 사용 | 사용 |

원본 label의 3·4·5번 위치가 예측 2·3·4번 위치와 대응

가상 토큰 ID · 예측 위치 t의 정답은 입력 위치 t+1

# 토큰 하나의 교차 엔트로피

## 손실 = −ln p(정답 토큰)

정답에 낮은 확률을 주면 더 큰 손실

여러 학습 위치의 손실을 모아 평균

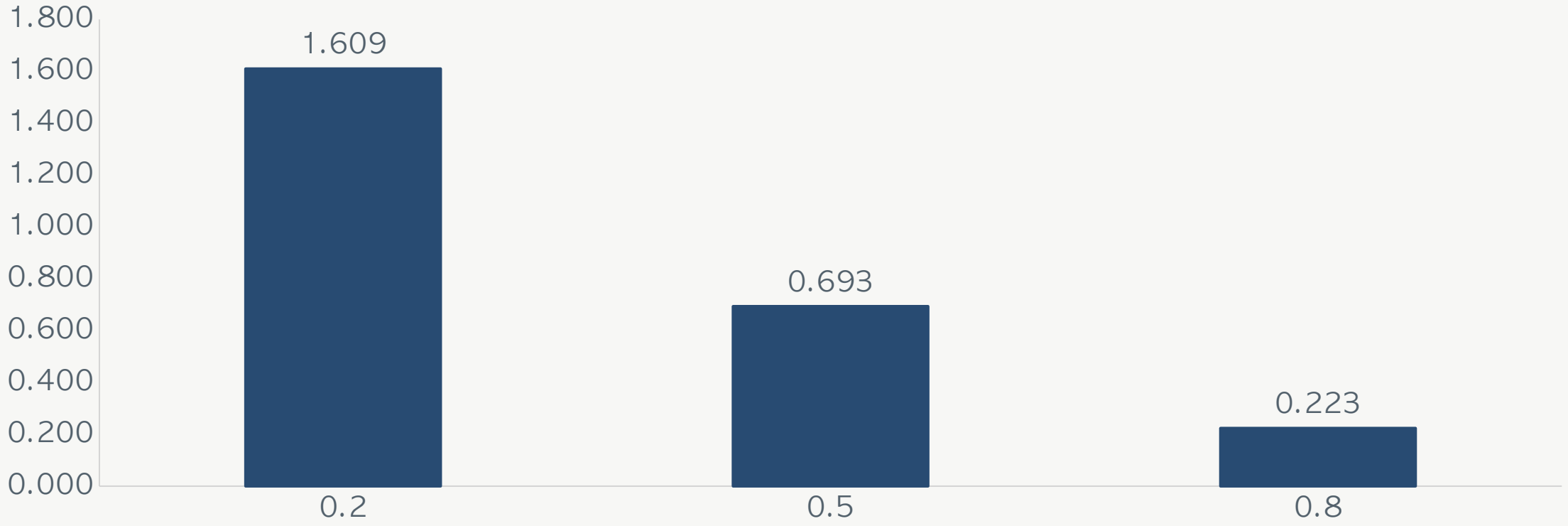
자연로그 기준 · 여기서는 토큰별 가중치를 동일하게 둔 설명용 계산

# 정답 확률과 손실의 수치

| 정답 토큰 확률 | −ln p | 해석 |
|---|---|---|
| 0.80 | 0.2231 | 정답에 높은 확률 |
| 0.50 | 0.6931 | 중간 확률 |
| 0.20 | 1.6094 | 정답에 낮은 확률 |

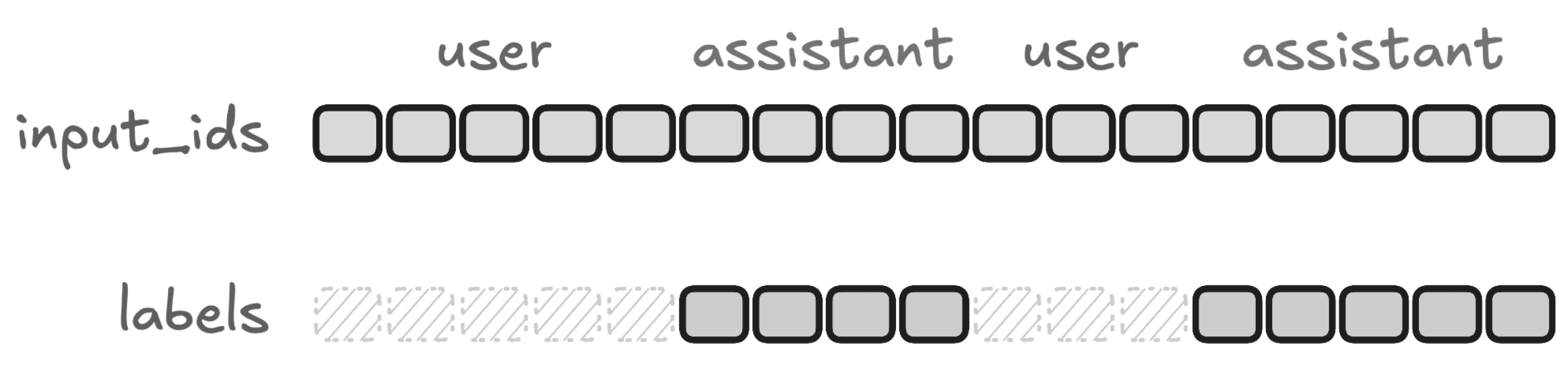
가상 확률의 계산값 · 실제 모델 측정 결과 아님

# 정답 확률별 손실 비교




가상 정답 확률 0.2·0.5·0.8 · 자연로그 손실 · 낮을수록 해당 정답 예측에 유리

# 답변 구간의 손실 마스킹




**user**

사용자 입력은 참고 문맥

**assistant**

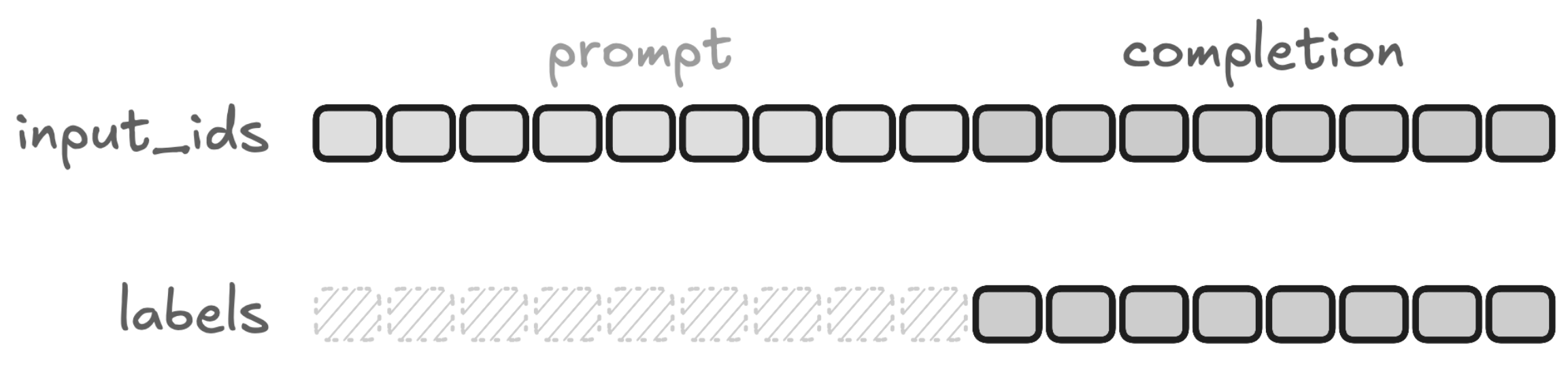
선택한 답변 토큰에 학습 신호

**빗금 구간**

손실 계산에서 제외한 위치

Hugging Face TRL · assistant-only loss · 모델별 템플릿 확인

# Prompt와 Completion의 분리




**prompt**

질문·지시·참고 문맥

**completion**

기대 답변

**확인**

답변 경계·종료 토큰·잘림

Hugging Face TRL · completion-only loss · 대화 역할 선택과 입력 형식 구분

# 마스킹과 Shift의 실행 예시

```
ids = np.array([10, 11, 12, 20, 21, 22])
labels = np.array([-100, -100, -100, 20, 21, 22])
targets = labels[1:]
active = targets != -100
assert targets[active].tolist() == [20, 21, 22]
```

6주차 Notebook · 가상 ID · -100은 손실에서 제외할 위치

# 마스킹 실패의 두 경우

## 입력까지 모두 학습

질문·문서 토큰의 손실까지 반영

목표와 데이터 형식에 맞는지 점검

## 답변까지 모두 제외

학습할 정답 위치가 0개

배치 손실과 마스크 수 사전 검사

어떤 마스크가 적절한지는 학습 목적·입력 형식에 따라 판단

# 길이 제한과 답변 누락

## 긴 문맥 때문에 정답 답변이 잘리는 사례

자른 뒤 남은 답변 토큰 수를 다시 확인

긴 사례·짧은 사례·경계 사례를 따로 점검

6주차 Notebook · 길이를 3·4·6으로 바꾸며 남은 학습 위치 비교

# 학습 데이터와 검증 설계

허가된 자료와 명확한 기대 답변

동일 사건의 표현을 한 분할에 모으는 원칙

# 데이터 품질의 검사 항목

| 검사 | 확인 대상 | 실패 예시 |
| --- | --- | --- |
| **필드** | 입력·답변·사건 ID | 빈 instruction |
| **출력** | 구문과 필수 키 | answer 키만 존재 |
| **중복** | ID·내용·같은 사건 | 표현만 바꾼 복사 |
| **권한** | 사용·보존·삭제 범위 | 고객 원문 무단 사용 |

형식 검사 통과 뒤에도 내용 검토와 사용 허가 필요

# 데이터 오류 주입과 탐지

```
data = examples()
bad = [dict(r) for r in data]
bad[0]["response"] = "{broken}"
report = audit(bad)
assert report["issues"][0]["kind"] == "invalid_output"
```

7주차 Notebook 01 · 정상 48건에 잘못된 JSON 한 건 주입

# 중복과 유사 사례의 차이

## 완전 중복

ID 또는 내용 해시로 탐지

중복 제거 기준과 이력 보존

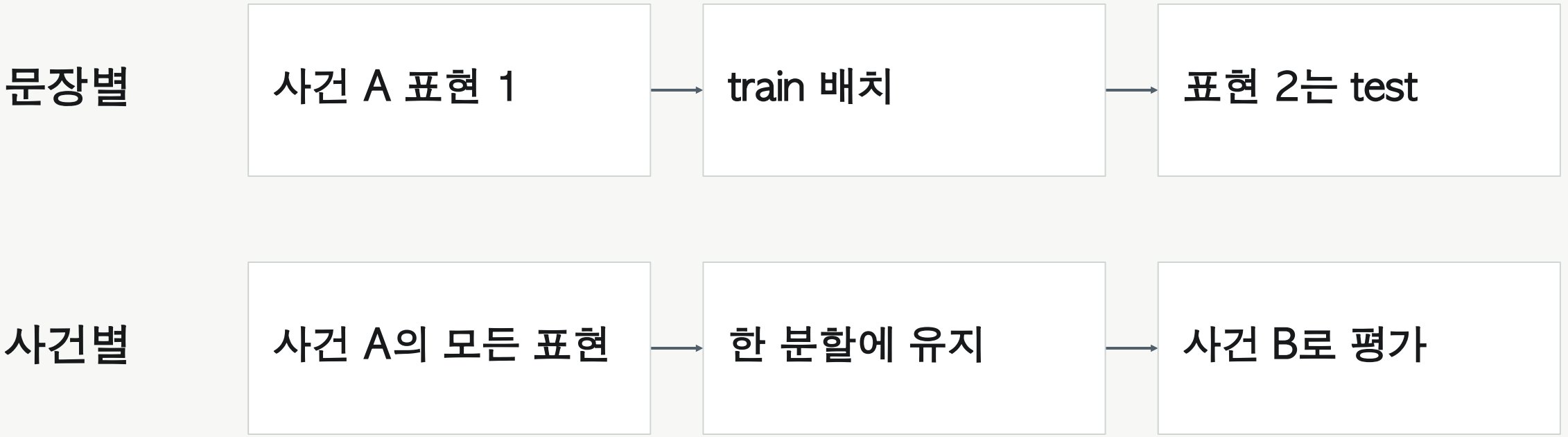
## 동일 사건의 다른 표현

문자열이 달라도 같은 정보 포함

사건 ID로 묶어 분할

정보 누출: 평가 대상 정보가 학습·설정 선택 과정에 섞이는 문제

# 문장 분할과 사건 분할




train: 학습용 · test: 최종 평가용 · 표현 암기와 새 사건 일반화의 구분

# 분할 이후의 누출 확인

```
splits = split_groups(data, seed=42)
assert not leakage(splits)
splits["test"].append(splits["train"][0])
assert leakage(splits)
```

7주차 Notebook 01 · 분할 이후 자료 추가 시에도 검사 반복

# 합성 사례 48건의 분할

| 분할 | 사건 수 | 문장 수 |
|---|---|---|
| train · 학습 | 16 | 32 |
| dev · 설정 선택 | 4 | 8 |
| test · 최종 보류 | 4 | 8 |

24사건 × 2표현 · seed=42 · 실제 고객 분포를 대표하지 않는 수업 자료

# 학습 설정과 관찰할 변화

| 설정 | 의미 | 점검할 현상 |
| --- | --- | --- |
| Learning rate | 한 번 갱신할 변화 크기 | 불안정·과도한 변화 |
| Epoch | 자료를 읽는 반복 횟수 | 학습 예시 편중 |
| Batch size | 한 번에 처리할 사례 수 | 메모리·학습 변동 |

단일 권장값보다 모델·자료·환경을 고정한 비교가 우선

# 학습 손실과 일반화

## 학습 데이터의 손실 감소

본 예시를 더 잘 예측하는 신호

외운 예시에 편중될 가능성 점검

## 보류 입력의 업무 품질

새 사건의 근거·형식·보류 확인

학습·설정 선택에 쓰지 않은 입력

오류 유형별 응답 원문과 실패 이유를 함께 보존

# SFT 준비의 완료 조건

| | | |
|---|---|---|
| 01 | **자료** | 허가·기대 답변·사건 단위 분할 |
| 02 | **입력** | 역할·종료 토큰·마스킹·잘림 |
| 03 | **비교** | 기준선과 보류 평가 계획 |

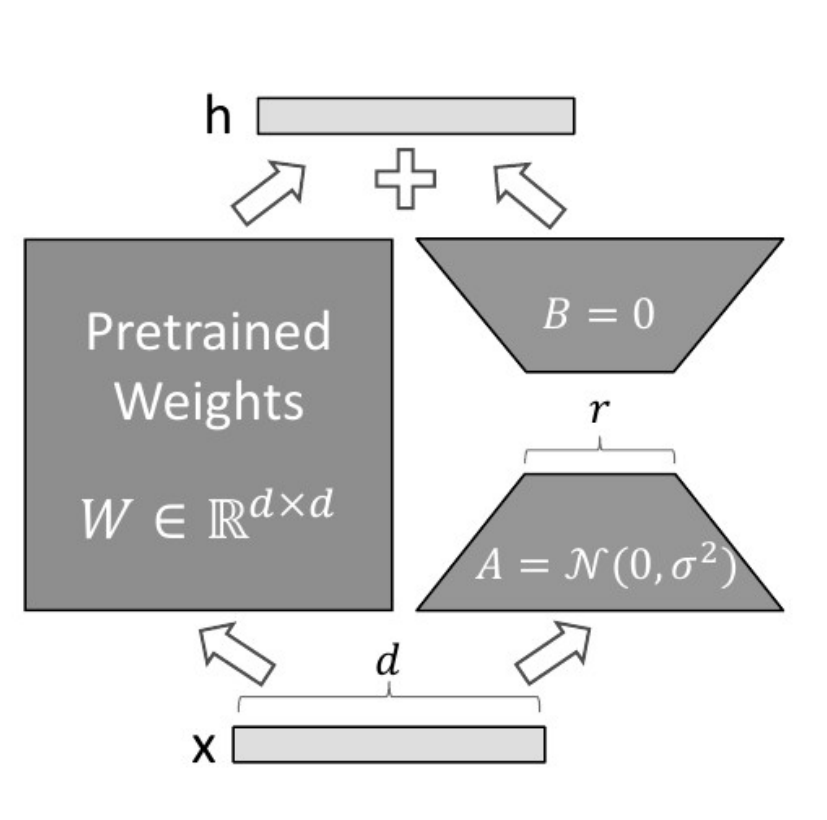
실제 GPU 학습 전 점검 목록 · 자동 검사와 내용 검토 병행

# LoRA의 구조

Low-Rank Adaptation · 저차원 행렬 기반 모델 적응

기존 가중치는 고정하고 작은 두 행렬을 학습

# LoRA 원 논문의 두 경로




## 고정 경로 W

기존 모델의 계산 유지

## 학습 경로 A·B

작은 변화량을 학습

## 합산

두 경로의 출력 결합

Hu et al., 2021, Figure 1 · 원본 도식 발췌·흑백 렌더링

# 가중치 변화의 표현

$$W' = W + (\alpha / r) \cdot BA$$

W: 고정 가중치 · A와 B: 학습 행렬

r: rank · $\alpha$ : 변화량 배율 설정

일반 LoRA scaling 기준 · rsLoRA 등 변형은 별도 설정 확인

# 입력과 행렬의 차원

01

## 입력 x

8 × 1

02

## A

2 × 8

03

## B

12 × 2

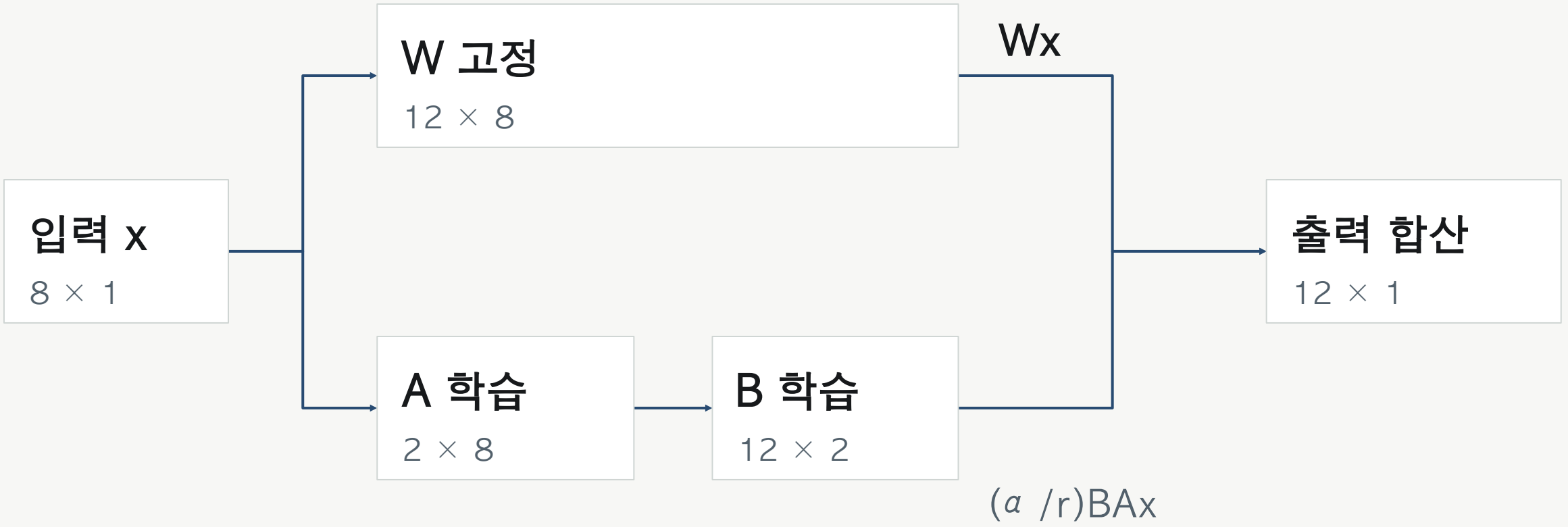
04

## 출력

12 × 1

r=2 예시 · BA는 12×8 · 기존 W와 동일한 차원

# 고정 경로와 학습 경로의 합산




동일 입력 x를 두 경로에 제공 · y = Wx + (α /r)BAx

# LoRA 파라미터 수의 계산

## 전체: d_out × d_in

---

LoRA: r × (d_in + d_out)

12×8, r=2이면 96개와 40개

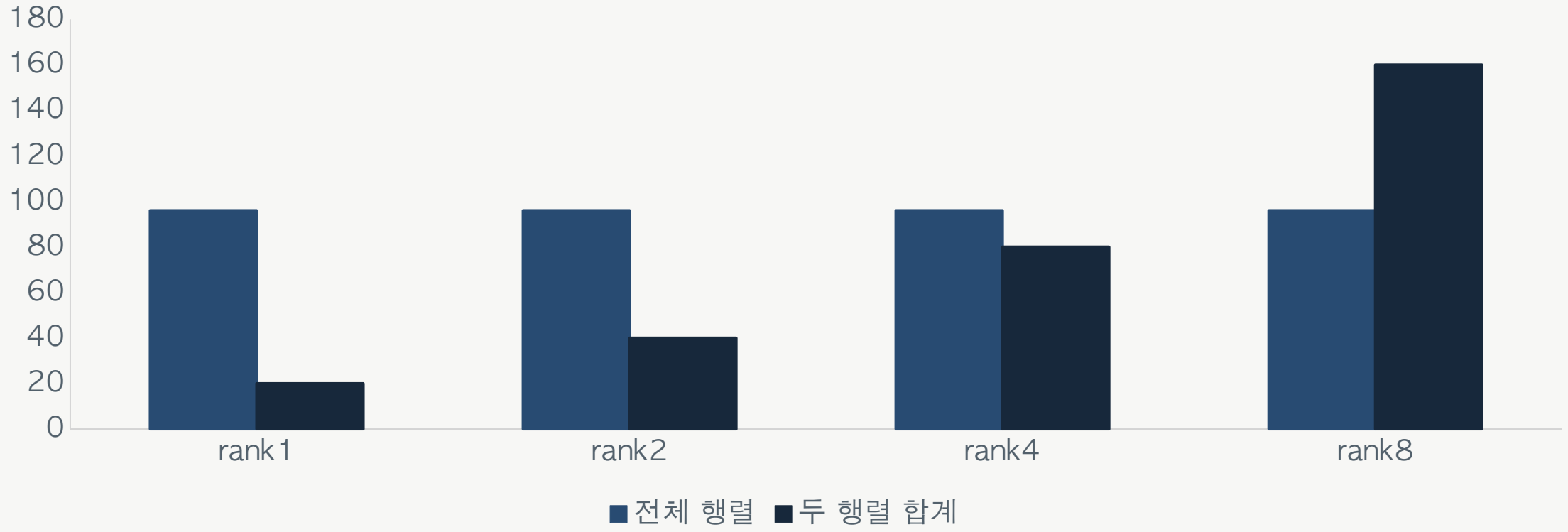
선형 층 하나의 A·B만 계산 · bias·다른 모듈·optimizer 비용 제외

# 작은 행렬의 rank별 비용

| rank | 전체 행렬 | A·B 합계 |
|---|---|---|
| 1 | 96 | 20 |
| 2 | 96 | 40 |
| 4 | 96 | 80 |
| 8 | 96 | 160 |

12×8 수업용 행렬 · 큰 rank에서는 두 행렬이 더 커질 수도 있음

# rank별 학습 파라미터



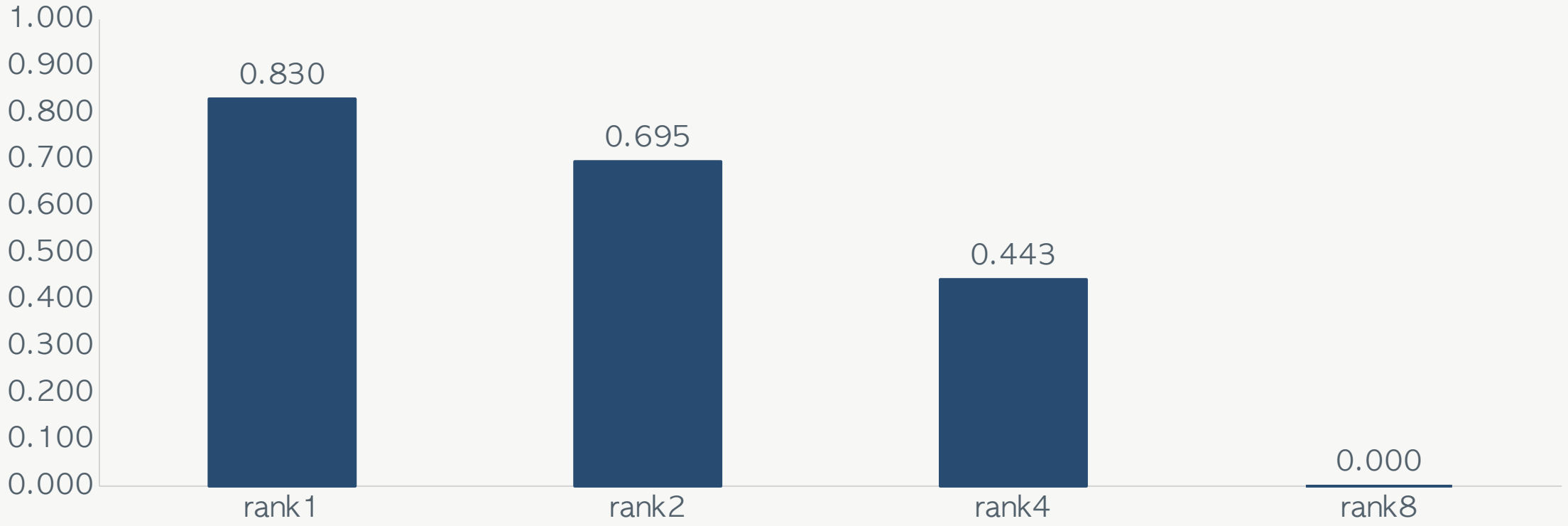

12×8 합성 행렬 · 전체 96개와 r(12+8)개 비교 · 메모리 실측 아님

# 4096차원 선형 층의 예시

| rank | A·B 파라미터 | 전체 대비 비율 |
|---|---|---|
| 8 | 65,536 | 0.3906% |
| 16 | 131,072 | 0.7813% |
| 32 | 262,144 | 1.5625% |

전체 4096×4096 = 16,777,216개 · 층 하나의 계산값 · 전체 모델과 구분

# rank별 행렬 근사 오차




7주차 Notebook 02 · 상대 Frobenius 오차 · 실제 LoRA 학습 결과 아님

# SVD 근사와 LoRA 학습

## 특이값 분해 실험

SVD: Singular Value Decomposition

완성된 행렬을 저차원으로 근사

## 실제 LoRA 학습

업무 데이터와 손실에서 A·B 갱신

새 입력의 서비스 품질을 별도 평가

행렬 복원 오차 감소를 언어 모델 품질 향상으로 해석하지 않음

# rank 선택의 실무 기준

## 동일 입력과 데이터로 후보 rank 비교

개발용 평가에서 품질·비용을 함께 기록

선택이 끝난 뒤 보류 평가셋으로 최종 확인

큰 rank가 항상 더 좋은 업무 품질을 보장하지 않음

# rank와 Alpha의 관계

| 설정 | $\alpha$ /r | 해석 |
|---|---|---|
| r=8, $\alpha$ =16 | 2 | 같은 BA에 2배 배율 |
| r=16, $\alpha$ =16 | 1 | rank와 배율이 함께 변경 |
| r=16, $\alpha$ =32 | 2 | 배율 유지, 용량은 변경 |

일반 scaling 수식의 비교 · 학습된 BA 자체도 달라질 수 있음

# 대상 모듈의 설정

```
r = 8
lora_alpha = 16
target_modules = ["q_proj", "v_proj"]
seed = 42
```

## 적용 위치와 실험 범위

대상 모듈 이름은 모델별 확인

다른 설정·환경도 함께 기록

설명용 설정 · 실제 모델 실행 전 구조와 구현 버전 재확인

# 초기 변화량과 원본 고정

```
A = rng.normal(size=(2, 8))
B = np.zeros((12, 2))
assert np.allclose(B @ A, 0)
before = W.copy()
adapted = W + B @ A
assert np.array_equal(W, before)
```

6주차 Notebook · A 무작위·B=0 초기화의 수치 확인

# Adapter 학습의 실행 흐름

01

## Base 로드

가중치 고정

02

## A·B 추가

학습 대상 확인

03

## 손실·갱신

데이터로 최적화

04

## 검증·저장

비교 후 보존

Adapter: Base 모델에 추가하는 학습 가능한 파라미터 묶음

# 재현 가능한 Adapter 묶음

| 구성 | 남길 정보 |
| --- | --- |
| Base | 모델 ID·revision·사용 조건 |
| Tokenizer | 버전·대화 템플릿 |
| Adapter | 가중치·rank·대상 모듈 |
| 실험 기록 | 자료 해시·설정·평가 원문 |

Adapter 파일만으로는 모든 재현 조건을 복원하기 어려움

# 분리 로드와 병합 배포

## Base와 Adapter 분리

같은 Base에 Adapter를 선택해 적용

조합 버전과 로딩 경로 관리

## 가중치 병합

지원하는 방식에서 변화량을 통합

양자화·정밀도 호환성과 결과 재평가

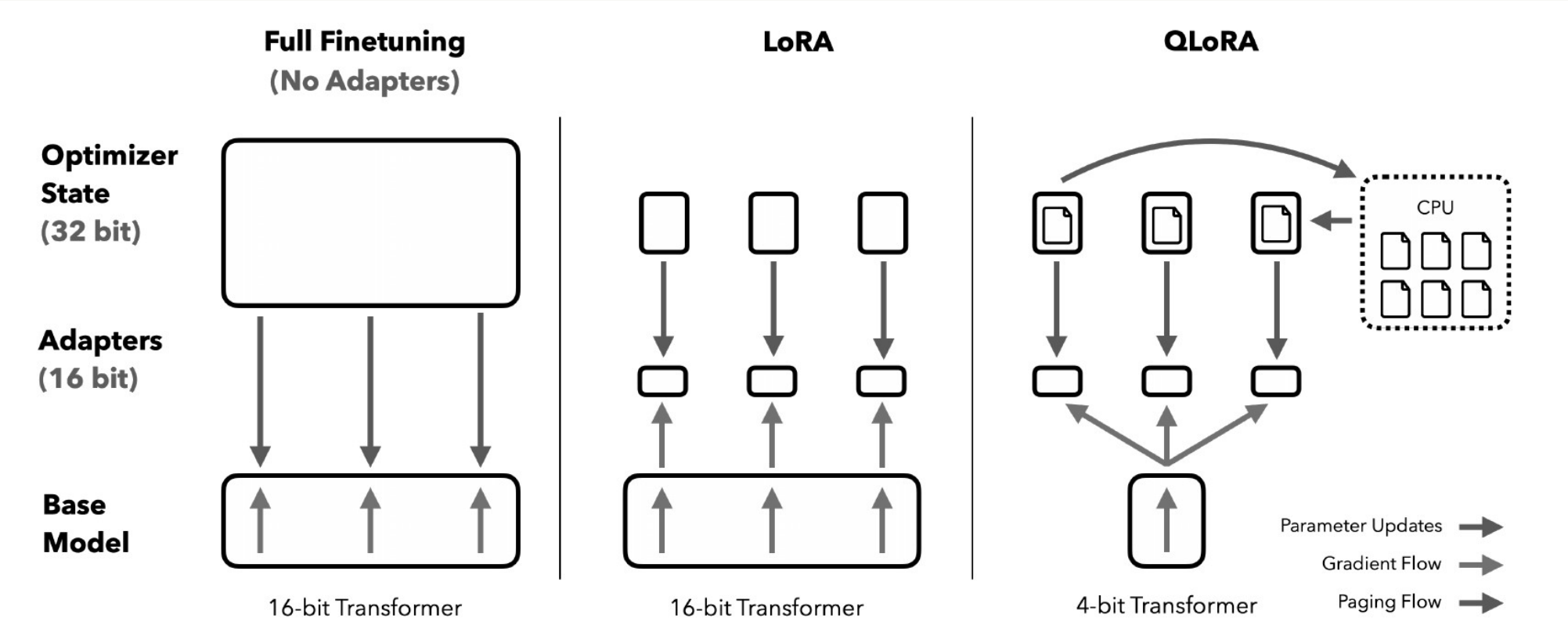
분리·병합 중 선택 · 속도 개선이나 수치 동일성을 미리 보장하지 않음

# QLoRA의 메모리 절약

Quantized Low-Rank Adaptation · 양자화 기반 LoRA

고정 모델의 저장 비용까지 줄이는 접근

# 전체 미세조정·LoRA·QLoRA




모델 가중치와 optimizer 상태 고정 모델과 학습 Adapter 4-bit 고정 모델과 Adapter

QLoRA Figure 1 · CC BY 4.0 · 흑백 발췌

# 저장 정밀도와 계산 정밀도

## 저장

고정 가중치를 낮은 bit로 압축

스케일·자료형 등 추가 정보 필요

## 계산

필요한 연산에서 복원해 계산

Adapter는 높은 정밀도로 학습

4-bit 저장만으로 모든 연산·활성값·optimizer가 4-bit가 되지는 않음

# 양자화의 작은 수치 예시

01

## 원래 값

−1.0부터 1.0

02

## 코드 선택

가까운 단계

03

## 압축 표현

코드와 스케일

04

## 복원

근삿값·오차

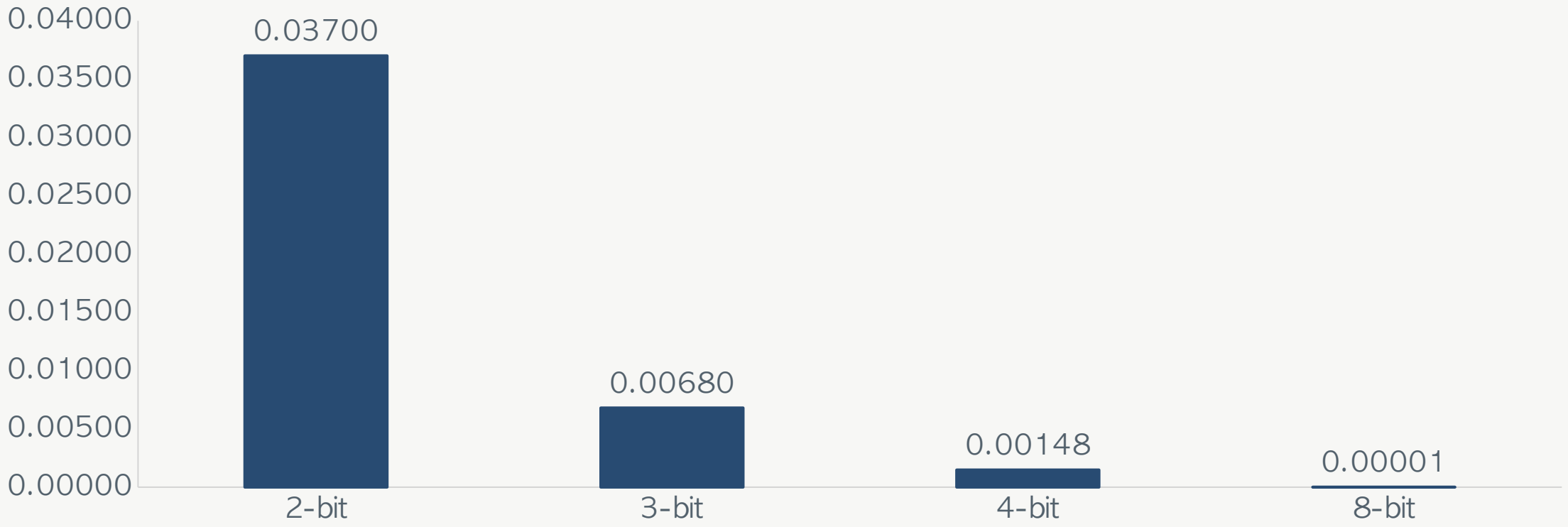
교육용 균등 양자화 · 실제 NF4 구현과 구분

# 균등 양자화와 복원

```
x = np.array([-1.0, -0.4, 0.2, 1.0])
levels = 2**2 - 1
q = np.rint((x + 1) * levels / 2)
restored = q * 2 / levels - 1
print(np.round(restored, 4))
```

6주차 Notebook · [-1,1] 범위의 2-bit 수업용 계산

# bit 수와 균등 양자화 오차




고정된 1,001개 값 〔-1,1〕 · 평균제곱오차 · NF4·모델 품질 측정 아님

# NF4와 균등 양자화

## 이번 수치 실험

일정 간격의 코드값 사용

bit 수와 복원 오차 관찰

## NF4

4-bit NormalFloat

정규분포 가중치를 고려한 코드값

균등 양자화 실험을 NF4 성능 실험으로 표시하지 않음

# Double Quantization의 대상

01

## 가중치 블록

값의 묶음

02

## 1차 양자화

코드·스케일

03

## 추가 양자화

스케일도 압축

04

## 복원

두 단계 정보 사용

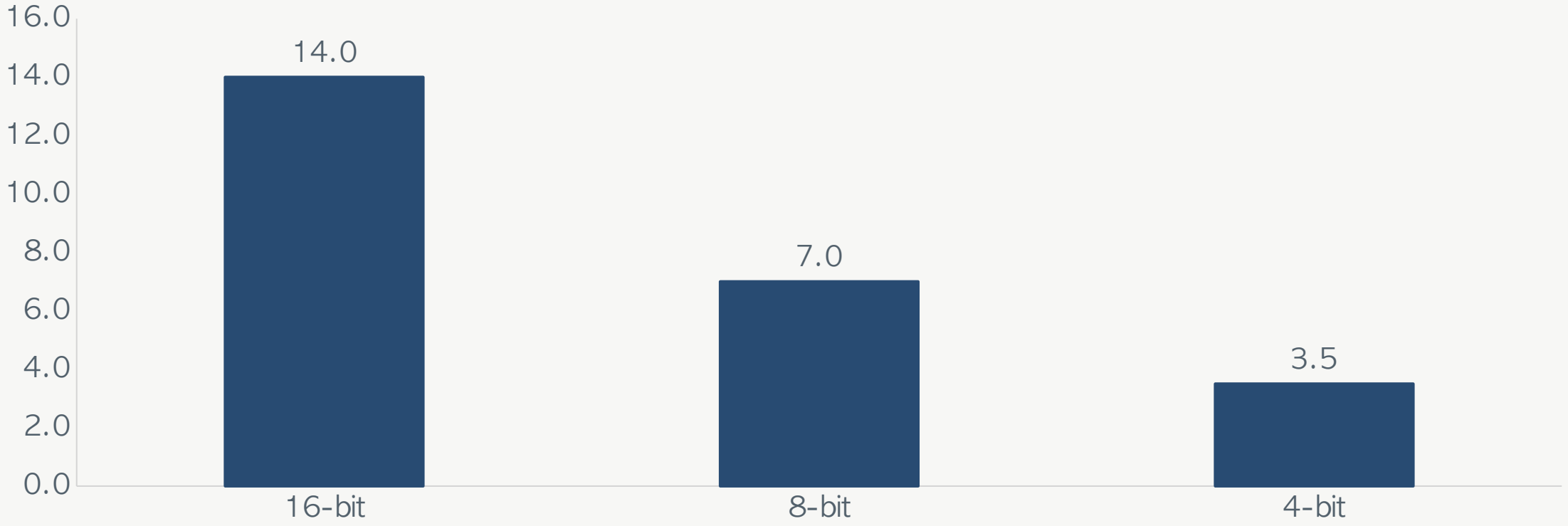
양자화 상수의 부가 저장량을 줄이는 방법

# Paged Optimizer의 역할

| 항목 | 읽는 방법 |
| --- | --- |
| **관리 대상** | optimizer 상태와 메모리 급증 |
| **접근** | CPU/GPU 메모리 이동 활용 |
| **남는 비용** | 이동 시간·하드웨어 지원 |
| **실행 확인** | 최대 메모리·처리 시간 동시 측정 |

운영체제 수준의 paging 개념과 연결 · 항상 빠른 학습을 보장하지 않음

# 7B 가중치의 원시 저장량




70억 파라미터 × bit/8 · 십진 GB · 메타데이터와 모든 학습 추가 비용 제외

# 총 학습 메모리의 구성

| 구성 | QLoRA에서 확인할 비용 |
| --- | --- |
| **고정 모델** | 양자화 가중치·스케일 |
| **학습 상태** | Adapter·gradient·optimizer |
| **활성값** | 입력 길이·batch·저장 방식 |
| **추가 공간** | 임시 버퍼·메모리 할당 여유 |

4-bit 가중치 3.5 GB 계산만으로 GPU 용량을 결정하지 않음

# 메모리 부족의 대응 순서

| 조정 | 관찰·주의할 점 |
| --- | --- |
| 입력 길이·batch 축소 | 잘림·학습 조건도 함께 변경 |
| Gradient accumulation | 작은 batch를 여러 번 누적 |
| Gradient checkpointing | 활성값 재계산과 시간 비용 |
| 양자화·paging 검토 | 지원 환경·품질·처리량 재확인 |

Out of Memory, OOM: 메모리 부족 오류 · 한 번에 한 조건을 변경

# 공통 실습과 GPU 학습 환경

## 공통 실습

Python 3.11 · 기존 NumPy 환경

모델·GPU·API 키 없이 개념 확인

## 실제 모델 학습

하드웨어·커널·의존성 별도 검증

보존용 legacy 환경을 공통 .venv와 분리

이번 강의의 수치 실험은 실제 QLoRA 학습 성공을 의미하지 않음

# 실습과 비교 평가

계산 결과와 업무 품질을 나누어 기록

실행 근거를 기획서의 기술 선택으로 연결

# VS Code 실습 준비

```
source .venv/bin/activate
python -m pip check
code .
```

기존 .venv가 없으면 lab/README.md의 설치 안내부터 진행

# 실습 파일과 장표의 연결

| 파일 | 실행할 내용 | 연결 장표 |
| --- | --- | --- |
| 6주차 Notebook 01 | 손실·마스킹·양자화 | 18–26, 51, 58–64 |
| 7주차 Notebook 01 | 오류·사건 분할·누출 | 28–33 |
| 7주차 Notebook 02 | rank·오차·파라미터 | 40–49 |

위에서 아래로 실행 · 예상값을 적고 한 변수 변경 · 산출물은 로컬 보관

# 통합 사례 · 문의 분류 서비스

| | | |
|---|---|---|
| 01 | **관찰** | JSON 누락과 근거 없는 단정을 따로 기록 |
| 02 | **가설** | 형식 반복 오류는 SFT 후보로 검토 |
| 03 | **검증** | 정상·예외·보류 사례를 같은 기준으로 비교 |

합성 기획 예시 · 최신 정책은 문서 검색·버전 관리와 함께 설계

# Base와 Adapter의 평가표

| 차원 | 같게 유지할 조건 | 남길 결과 |
| --- | --- | --- |
| **형식·근거** | 입력·문서·출력 규칙 | 응답 원문·실패 이유 |
| **비용** | 기기·길이·반복·예열 | 시간·최대 메모리 |
| **재현** | 모델·Tokenizer·생성 | 버전·Adapter·자료 해시 |

train loss 하나로 배포 결정 불가 · 보류 입력과 오류 유형별 비교

# 비교 결과와 채택 판단

| | | |
|---|---|---|
| 01 | **분리** | 형식·근거·보류·운영 비용을 각각 확인 |
| 02 | **검토** | 평균뿐 아니라 실패 사례와 표본 수 검토 |
| 03 | **결정** | 개선 근거·남은 위험·되돌릴 조건 기록 |

정해진 업무 평가를 실행하기 전에는 개선 성과를 단정하지 않음

# 형식 개선 이후의 회귀 오류

## 좋아진 부분

JSON 필수 키 누락 감소

동일 입력에서 형식 기준 통과

## 나빠진 부분

근거 없는 답변이나 과도한 거절

실패 원문 검토 후 학습·자료 재점검

가상 결과 시나리오 · 실제 실행 측정값이 아닌 판단 연습

# 핵심 확인 질문과 답

| 질문 | 확인할 답 |
| --- | --- |
| SFT와 LoRA는 대안인가 | 학습 목적과 갱신 방법으로 결합 가능 |
| 4-bit면 메모리도 1/4? | 활성값·학습 상태·부가 정보는 별도 |
| SVD 오차와 서비스 품질? | 행렬 실험과 업무 평가를 분리 |
| 손실만으로 배포 가능한가 | 보류 입력·실패·비용·복구 검토 |

기획서에는 기술명보다 해결할 오류와 확인할 실험을 명시

# 출처와 복습 경로

| 자료 | 원문·실행 위치 |
| --- | --- |
| SFT 공식 도식 | https://huggingface.co/docs/trl/sft_trainer |
| LoRA 원 논문 | https://arxiv.org/abs/2106.09685 |
| QLoRA 원 논문 | https://arxiv.org/abs/2305.14314 |
| 수치·데이터·행렬 실습 | week06/lab/notebooks · week07/lab/notebooks |

추가 URL·외부 이미지 이용 범위: lecture/assets/SOURCES.md